

Introduction to C Programming

A Beginner's Course

STUDENT MANUAL

A complete, offline, printable reference for every module of the course —
from your first compiled program through the team capstone project.

Table of Contents

Welcome / How This Course Works

Welcome! This course takes you from **never having written a line of code** to **building a multi-file C project with pointers, structs, and file I/O, tracked in Git and submitted through GitHub Classroom**. No prior programming experience is assumed.

Built to work with or without a lecturer in the room

Every lesson is written so you can teach yourself from it directly — the *why* always comes before the *how*, every claim is backed by code you actually run, and nothing assumes a lecturer will fill in a gap out loud. If you have an instructor, they'll add depth, war stories, and live debugging. If you don't, you can still get every concept from the page.

Every module starts with a hook

Before any theory, every module opens with a **Hook** — a short, genuinely fun demo or puzzle you run *first*, before you understand how it works. Curiosity ("wait, why did that happen?") is the entire teaching strategy — it's a far stronger reason to keep going than "you'll need this later." By the end of each module you'll understand exactly how your own hook worked.

How this course is built

Every module (after Setup) follows the same shape:

1. **README.md** — the lesson. A hook first, then concepts explained in plain language, with common beginner mistakes called out explicitly.
2. **examples/** — small, runnable `.c` files that demonstrate exactly one idea at a time. Compile and run every single one yourself. Reading code you didn't type teaches you far less than watching your own build succeed (or fail) at the terminal.
3. **exercises/** — practice problems with starter files containing `TODO` markers. A `solutions/` folder sits next to them — try for at least 15 minutes before you look.
4. **project/** — a small, fun program that only uses what you've learned *so far*. This is the payoff for the module — something you'd actually want to show a friend.

Roadmap

#	Module	You will be able to...
0	Setup: Tools, Git & GitHub Classroom	Install a compiler, use VS Code, and submit work through GitHub Classroom
1	Basics of Programming	Explain how a C program becomes a running program, and write/compile your first one
2	Data Types & Operators	Choose the right type for a piece of data and compute

#	Module	You will be able to...
		with it correctly
3	Variables & Input/Output	Store data, name it well, and write programs that talk back to the user
4	Control Structures	Make decisions (<code>if/switch</code>) and repeat work (loops)
5	Arrays & Strings	Store and process collections of values and text
6	Functions	Break a program into reusable, named, testable pieces
7	Pointers	Work directly with memory addresses — the idea that makes C, C
8	Structs & File I/O	Design your own data records and make programs remember things after they close
9	Capstone Project	Combine everything into one real program, built in a team and submitted via GitHub Classroom

Ground rules for how we'll work

- **Type the code yourself.** Copy-pasting defeats the purpose — your fingers and your mistakes are how the syntax sticks.
- **Read every compiler error fully,** top to bottom, before asking for help. The first error in a long list is usually the real one; everything after it is often noise caused by that first mistake.
- **Compile often.** Write 3–5 lines, then compile. Don't write 100 lines and try to debug them all at once.
- **Commit often.** After finishing an example, exercise, or project step, commit it. Module 0 shows you exactly how. Small, frequent commits are a professional habit you're building from day one — and they're also your evidence of steady progress once the capstone is a team project.

How work gets submitted

Every exercise, module project, and the capstone are submitted through **GitHub Classroom**, not by email or file upload:

- Individual modules (0–8): you accept a per-student assignment link once (Module 0 walks through this), which gives you your own private repo. Commit and push your work there as you go — the last commit before a deadline is what gets graded.

- The capstone (Module 9): a **group assignment** — GitHub Classroom creates one shared repo per team. See **Module 9** for exactly how teams are formed and how many people are on yours.

Prerequisites

None, other than curiosity and a willingness to make (and fix) mistakes. Mistakes are not a sign you're bad at this — they're the main way anyone learns to program. Start with **Module 0: Setup**.

Module 0 — Setup: Tools, Git & GitHub Classroom

Hook: your first build, before you understand a single line

Before anything else, let's prove the machinery works. Create a file called `hook.c` anywhere on your computer and paste this in exactly:

```
#include <stdio.h>

int main(void) {
    int year = 2026;
    printf("If you started today, by %d you could have written your\n", year);
    printf("own game, your own tool, or your own excuse for why the\n");
    printf("bug definitely isn't your fault. Let's find out.\n");
    return 0;
}
```

Once your compiler is installed below, run:

```
gcc hook.c -o hook
./hook
```

You just turned text into a running program on your own machine. You don't understand `#include`, `printf`, or `return` yet — that's fine, that's the entire rest of this course. Keep that feeling of "it worked and I don't fully know why" in mind; every module from here replaces one piece of "I don't know why" with "I know exactly why."

1. Install a C compiler

C needs to be **compiled** — translated into machine code before it runs (more on this in Module 1). You need a compiler on your machine.

macOS

```
xcode-select --install
```

This installs Apple's Command Line Tools, which includes `clang` (a C compiler). This course uses `gcc` as the command name throughout — on macOS, `gcc` is typically aliased to `clang` already. Check with:

```
gcc --version
```

Windows

Install **WSL (Windows Subsystem for Linux)** — open PowerShell as Administrator and run `wsl --install`, then restart. Inside the Ubuntu terminal that opens, run:

```
sudo apt update && sudo apt install build-essential -y
```

This gives you `gcc`, `make`, and a real Linux terminal — the same environment your instructor and classmates are using, which avoids "works on my machine" problems.

Linux

```
sudo apt install build-essential -y      # Debian/Ubuntu
sudo dnf groupinstall "Development Tools" # Fedora
```

Verify on any platform:

```
gcc --version
```

You should see a version number, not "command not found."

2. Install VS Code

Download from code.visualstudio.com. Then install two extensions from the Extensions panel (Cmd/Ctrl+Shift+X):

- **C/C++** (by Microsoft) — syntax highlighting and IntelliSense.
- **Code Runner** (by Jun Han) — lets you run a `.c` file with one click.

3. Install and configure Git

Git is almost certainly already installed (`git --version` to check). If not, install it from git-scm.com. Then tell Git who you are — this name and email get attached to every commit you make:

```
git config --global user.name "Your Full Name"
git config --global user.email "the-email-you-use-for-github@example.com"
```

4. Create a GitHub account

If you don't already have one, sign up at github.com. Use a professional-looking username — this account will hold your work long after this course ends, and employers do look.

5. Accept your first GitHub Classroom assignment

This course does **not** collect work by email or file upload — everything goes through **GitHub Classroom**.

1. Your instructor shares an **assignment invite link** for Module 0 (in a codespace, LMS post, or class chat).
2. Open the link and sign in with your GitHub account.
3. Click **Accept this assignment**. GitHub Classroom creates a private repository just for you (something like `module-0-setup-yourusername`), copied from a starter template.
4. Wait a few seconds for "Your assignment repository has been created" and click through to it.

6. Clone your repository and make your first commit

On your computer, in a terminal, navigate to where you keep your code and clone the repo GitHub Classroom just created for you:

```
git clone https://github.com/YOUR-CLASSROOM-ORG/module-0-setup-
yourusername.git
cd module-0-setup-yourusername
```

Open the folder in VS Code (code . from that terminal, or File → Open Folder). Add a `hook.c` file with the hook program from the top of this page (personalize the message if you like), then:

```
git add hook.c
git commit -m "Complete Module 0 setup"
git push
```

Refresh the repository page on GitHub — you should see your file and your commit. **This is your workflow for every module from now on:** edit code locally, `git add` the files you changed, `git commit` with a short description, `git push` to send it to your GitHub Classroom repo. Nothing is submitted until it's pushed.

Common setup problems

- `gcc: command not found` — the compiler isn't installed, or your terminal was opened before installation finished. Close and reopen the terminal.
- `git push` asks for a password and rejects it — GitHub no longer accepts account passwords for this. Use a **personal access token** as the password, or set up SSH keys (ask your instructor which your class uses).
- Nothing shows up on GitHub after `git push` — check `git status` first; if it says "nothing to commit," you likely forgot `git add` before `git commit`.

Try it yourself

1. Confirm `gcc --version` and `git --version` both work.
2. Accept the Module 0 GitHub Classroom assignment.
3. Clone it, add your `hook.c`, commit, and push.
4. Confirm your commit appears on GitHub.

Next: **Module 1 — Basics of Programming.**

Module 1 — Basics of Programming

Hook: the world's smallest fortune teller

Compile and run this exactly as written — don't read ahead first:

```
#include <stdio.h>

int main(void) {
    printf("You will encounter a strange creature today.\n");
    printf("It has six legs, compound eyes, and excellent opinions.\n");
    printf("Its name is: ");
    printf("your cat.\n");
    return 0;
}
```

```
gcc fortune.c -o fortune
./fortune
```

Four `printf` calls, one program, one joke that lands because of the line break placement. By the end of this module you'll understand exactly why that third `printf` doesn't start a new line, and you'll build your own version of this in the module project.

What is a program, really?

A computer only understands one language: electrical signals interpreted as numbers — **machine code**. Nobody writes programs directly in machine code anymore. Instead, we write in a **language humans can read** (like C), and a program called a **compiler** translates it into machine code for us.

you write		compiler translates		computer runs
hello.c	--->	(gcc)	--->	hello
(C source)				(machine code)

This is exactly what you did in Module 0 when you ran `gcc hook.c -o hook`. Every C program you ever write follows this same pipeline: **source code** → **compiler** → **executable** → **run**.

C is a **compiled language**. Contrast this with languages like Python, which are **interpreted** — run line-by-line by another program every time, with no separate "build" step. This is why compiled programs tend to run faster (all the translation work happens once, ahead of time) but have an extra step (you must recompile after every change).

Anatomy of a C program

Look at the smallest complete, useful C program:

```
#include <stdio.h>

int main(void) {
    printf("Hello, C!\n");
    return 0;
}
```

Let's take it apart line by line.

```
#include <stdio.h>
```

C's core language is small — even printing to the screen isn't built in. Instead, functionality lives in **libraries** you pull in with `#include`. `stdio.h` ("standard input/output header") gives you `printf` (for printing) and `scanf` (for reading input, coming in Module 3). This line must come before you use anything from that library.

```
int main(void) { ... }
```

Every C program has exactly one `main` function — it's the **entry point**: the first code that runs when your program starts. `int` means `main` reports back a whole number when it finishes (see `return 0` below). `void` between the parentheses means "this version of `main` takes no inputs" — you'll see a different form of `main` if you ever handle command-line arguments, but this is the one you'll use for now. The `{` and `}` mark where `main`'s body begins and ends — everything a function does lives between its braces.

```
printf("Hello, C!\n");
```

- `printf` — "print formatted" — a function from `stdio.h` that writes text to the screen.
- `"Hello, C!\n"` — a **string literal**: literal text, always in double quotes. `\n` is a single special character meaning "newline" — it's what moves output to the next line. Without it, the next `printf` would continue on the *same* line (exactly what the `hook` program exploited).
- `;` — a **semicolon** ends every statement in C. Forget one and the compiler will complain — usually about the *next* line, which confuses beginners. If an error message makes no sense, check the line *above* it for a missing semicolon first.

```
return 0;
```

Tells whoever ran the program "I finished successfully." `0` conventionally means success; any other number signals some kind of error occurred. You won't use this deliberately much yet, but it must be there in `main`.

Statements, whitespace, and comments

- A **statement** is one instruction, ending in `;`. C doesn't care about line breaks or indentation the way Python does — they're purely for humans to read the code. *Always* indent consistently anyway; VS Code's C/C++ extension does this for you automatically.
- **Comments** are notes for humans that the compiler ignores entirely:

```
// A single-line comment — everything after // on this line is ignored.

/* A multi-line comment.
   Everything between /* and */ is ignored, across as many lines as you like.
*/
```

Use comments to explain *why* something is done a non-obvious way — not to restate what the code already says.

Compiling and running — the two ways you'll use in this course

Terminal (do this to really understand it):

```
gcc myfile.c -o myfile
./myfile
```

VS Code Code Runner (fast iteration once you're comfortable): open the `.c` file and click the  button in the top-right corner, or press `Ctrl+Alt+N`.

Both do the same thing. Use the terminal version whenever you want to be sure you understand exactly what's happening — it's also what you'll use on any computer that doesn't have VS Code.

Errors are normal — read them, don't fear them

Try compiling this broken version (missing semicolon):

```
#include <stdio.h>

int main(void) {
    printf("Oops\n")
    return 0;
}
```

gcc will report something like: `error: expected ';' before 'return'`

That's the compiler pointing almost exactly at the mistake. Reading compiler errors is a skill — you'll get fast at it. The habit that matters most right now: **fix the first error first, then recompile**. One missing semicolon can produce five confusing-looking errors below it; they usually all disappear once you fix the real one.

Common beginner mistakes

- Forgetting the semicolon at the end of a statement.
- Mismatched `{` and `}` — VS Code highlights the matching brace when your cursor is next to one; use it.
- Forgetting `#include <stdio.h>` and then using `printf` — some compilers will still let this slide with a warning, but always include it; never rely on it being implicit.
- Forgetting `\n` and being confused why two `printf` calls' output ran together on one line.
- Typos in `printf` (e.g. `Printf`, `PRINTF`) — C is **case-sensitive**.

Try it yourself

1. Work through every file in **examples/**, compiling and running each one.
2. Complete **exercises/exercisel.c**.
3. Build the **module project**.
4. Commit and push your work (see Module 0 §6 if you need a refresher):

```
git add .
git commit -m "Complete Module 1: basics of programming"
git push
```

Next: **Module 2 — Data Types & Operators.**

Module 2 — Data Types & Operators

Hook: why did math just break?

Compile and run this exactly as written — don't read ahead first:

```
#include <stdio.h>

int main(void) {
    printf("Splitting 7 slices of pizza among 2 people: %d slices each\n", 7 /
2);

    unsigned char tinyBox = 255;
    tinyBox = tinyBox + 1;
    printf("One more than 255 in a box that only holds 0-255: %d\n", tinyBox);

    return 0;
}
```

```
gcc hook.c -o hook
./hook
```

You should see 3 slices each (not 3.5 — did someone eat half a slice?), and then 0 where you'd expect 256. Neither of these is a bug in your program. Both are C doing *exactly* what it was told, using the rules for **data types** and **operators** you're about to learn. By the end of this module you'll know precisely why 7 divided by 2 lost its remainder, and why a number can "wrap around" back to zero — and you'll use both facts on purpose in this module's project.

Why data types exist

Every piece of data in a C program lives in memory as a fixed-size box of bits. A **data type** tells the compiler two things: how big that box is, and how to interpret the bits inside it. Get the type wrong for what you're storing, and you get wrong (but silent!) answers — like the hook just showed you. This is different from what you're used to if you've ever used a calculator or spreadsheet, where numbers "just work." In C, you choose the box size up front.

The basic types

```
#include <stdio.h>

int main(void) {
    int wholeNumber = 42;
    float singlePrecision = 3.14f;
    double doublePrecision = 3.14159265358979;
    char letter = 'A';

    printf("%d\n", wholeNumber);
    printf("%f\n", singlePrecision);
    printf("%f\n", doublePrecision);
    printf("%c\n", letter);

    return 0;
}
```

- **int** — whole numbers, positive or negative, no decimal point: 42, -7, 0.
- **float** — numbers with a decimal point, stored with limited precision (about 6-7 reliable digits). Written with an `f` suffix in code, e.g. 3.14f.
- **double** — also decimal numbers, but roughly twice the storage of `float` and about twice the precision. This is C's *default* choice for decimal numbers when you're not tight on memory — prefer it over `float` unless you have a specific reason not to.
- **char** — a single character, written in **single** quotes: 'A', '7', '\$'. Under the hood it's just a small integer (each character has a numeric code), which is why arithmetic on `char` values is legal, if unusual, in C.

You'll also see `short` and `long` mentioned in other people's code — they're `int` variants with smaller or larger ranges. As a beginner you'll rarely need them; stick to `int`, `float`, `double`, and `char` for now.

This line is new too: `int wholeNumber = 42;` **declares** a variable (reserves a labeled box of memory of that type) and **initializes** it (puts a value in the box) in one step. We're using variables here purely as a way to hold values so we can inspect types and operators — Module 3 goes much deeper on naming them well, `const`, and reading values in from the user with `scanf`.

Picking the right format specifier

`printf` needs to be told what type you're handing it, using a **format specifier** inside the string:

Type	Specifier
<code>int</code>	<code>%d</code>
<code>float/double</code>	<code>%f</code>
<code>char</code>	<code>%c</code>

Get this wrong (e.g. `%d` for a `float`) and you'll get garbage output — the compiler often won't stop you, so double-check it yourself. There's also `%s` for printing strings of text, but strings are a Module 5 topic — you won't need it yet.

sizeof — how big is that box, really?

`sizeof` tells you exactly how many bytes a type (or a variable) occupies in memory:

```
printf("%zu\n", sizeof(int));
printf("%zu\n", sizeof(wholeNumber));
```

Note the format specifier: `sizeof` produces a special unsigned type, and `%zu` is the correct specifier for it — not `%d`. On a typical machine you'll see `int` and `float` at 4 bytes, `double` at 8 bytes, and `char` at exactly 1 byte (by definition — `char` is always 1 byte in C). This is also the mechanism behind the hook: `unsigned char` only has 1 byte to work with, giving it 256 possible values (0-255). Add 1 past the top of its range and there's nowhere for the extra bit to go — it silently **wraps around** back to 0.

Arithmetic operators — and the integer division trap

C gives you the five arithmetic operators you'd expect: `+` `-` `*` `/` `%`. The first three behave the way you'd guess. The last two need care.

```
int apples = 7;
int friends = 2;

printf("%d\n", apples / friends);      // 3 -- NOT 3.5
printf("%d\n", apples % friends);     // 1 -- the remainder
```

When both operands of `/` are integers, C performs integer division: the decimal portion is discarded entirely, not rounded. `7 / 2` is 3, full stop. This is exactly what the hook demonstrated, and it is *probably the single most common source of silently-wrong output* for beginners in C. To get a true decimal answer, at least one operand must be a `float` or `double` — which brings us to conversion and casting.

`%` (modulo) gives you the remainder left over from integer division: `7 % 2` is 1, because 7 divided by 2 is 3 remainder 1. It only works on integer types.

Type conversion and casting

Implicit conversion happens automatically when an expression mixes types — C "promotes" the smaller/less-precise type up to match:

```
int wholeScore = 7;
float bonus = 2.5f;
float total = wholeScore + bonus;    // wholeScore is promoted to float first
```

This is convenient, but it doesn't save you from the integer division trap above — that trap only happens *before* any promotion, when **both** operands are already integers. To fix it, you need an **explicit cast**: write the target type in parentheses right before the value you want converted.

```
int a = 7;
int b = 2;

printf("%d\n", a / b);           // 3 -- integer division
printf("%f\n", (float)a / b);   // 3.500000 -- a is converted to float FIRST
```

`(float)a` converts just `a` to a float *before* the division runs. Since one operand is now a float, the whole division becomes float division. This one technique — casting one operand before dividing — is the fix for the hook's first surprise, and you'll use it constantly.

Relational and logical operators

Relational operators compare two values and produce 1 (true) or 0 (false) — C has no separate boolean type for this:

```
int age = 20;

printf("%d\n", age == 20); // 1 (equal to)
printf("%d\n", age != 20); // 0 (not equal to)
printf("%d\n", age > 18);  // 1 (greater than)
printf("%d\n", age < 18);  // 0 (less than)
printf("%d\n", age >= 20); // 1 (greater than or equal to)
printf("%d\n", age <= 19); // 0 (less than or equal to)
```

Watch out: `==` (comparison) and `=` (assignment) are different operators that look almost identical — mixing them up is a classic bug you'll meet again in Module 4.

Logical operators combine 0/1 results: `&&` (and — true only if both sides are true), `||` (or — true if either side is true), and `!` (not — flips true to false and vice versa):

```
printf("%d\n", (age > 18) && (age < 65)); // 1 -- both true
printf("%d\n", (age < 18) || (age > 65)); // 0 -- both false
printf("%d\n", !(age == 20));           // 0 -- flips 1 to 0
```

Right now we're just printing what these operators produce. Using them to actually branch your program's behavior (`if` statements) is Module 4's job — for now, just get comfortable with the fact that a comparison *is* a value in C: 0 or 1, ready to be printed, stored, or combined with others.

Operator precedence

Just like in math class, `*` and `/` run before `+` and `-`:

```
printf("%d\n", 2 + 3 * 4); // 14, not 20
printf("%d\n", (2 + 3) * 4); // 20 -- parentheses run first
```

You don't need to memorize a full precedence table right now. The practical rule: **when you're not 100% sure how an expression will be evaluated, add parentheses.** It costs nothing, makes your intent obvious to anyone reading the code, and removes all doubt.

Common beginner mistakes

- Assuming `int / int` gives a decimal answer. It doesn't — `7 / 2` is 3. Cast at least one operand to `float` or `double` if you need a real decimal result.
- Using the wrong `printf` format specifier for a type (e.g. `%d` for a `float`) — this often doesn't error, it just prints garbage.
- Using `%d` for the result of `sizeof` instead of `%zu`.
- Confusing `=` (assignment) with `==` (comparison).
- Forgetting single quotes for `char` literals (`'A'`, not `"A"` — double quotes are for strings, which are a different, later topic).
- Casting the wrong operand or casting too late — `(float)(a / b)` still does integer division first and only converts the already-wrong answer; you must cast **before** the division happens, e.g. `(float)a / b`.

Try it yourself

1. Work through every file in **examples/**, compiling and running each one. Predict the output before you run it, then check.
2. Complete **exercises/exercisel.c**.
3. Build the **module project**.
4. Commit and push your work:

```
git add .
git commit -m "Complete Module 2: data types and operators"
git push
```

Next: **Module 3 — Variables & Input/Output**.

Module 3 — Variables & Input/Output

Hook: the instant mind reader

Compile and run this exactly as written — don't read ahead first:

```
#include <stdio.h>

int main(void) {
    int age;

    printf("Think of your age. Don't tell anyone. Just type it in: ");
    scanf("%d", &age);

    int birth_year_guess = 2026 - age;
    int days_alive_guess = age * 365;

    printf("\nI sense... you were born around the year %d.\n",
    birth_year_guess);
    printf("You have been alive for roughly %d days.\n", days_alive_guess);
    printf("I am basically a wizard.\n");

    return 0;
}
```

```
gcc mindreader.c -o mindreader
./mindreader
```

Type in your age and watch it "read your mind." There's no trick — just a variable holding the number you typed, and two lines of arithmetic. By the end of this module you'll understand every piece of this program: how `age` stores what you typed, what that `&` before it is doing, and how to build interactive programs like this from scratch. You'll build a bigger version of it in the module project.

What is a variable, really?

Every program so far has only ever printed text that was baked into the code itself. That's not very useful — a real program needs to work with information that changes: a score, a price, an age typed in by whoever is running it. A **variable** is a named box in the computer's memory that holds a value your program can read and change while it runs.

```
int age = 16;
^   ^   ^
|   |   +-- the value stored in the box right now
|   +-- the name you use to refer to the box
+-- the type of value the box is allowed to hold (Module 2)
```

You've already used types (`int`, `float`, `double`, `char`) and literal values in Module 2. A variable is what lets a value have a *name* you can reuse, print, and update throughout a program instead of typing the literal value over and over.

Naming variables: rules and conventions

C enforces a small set of **rules** for names (the compiler will reject anything that breaks them):

- Names can contain letters, digits, and underscores (`_`) only.
- A name **cannot start with a digit** (`1st_place` is illegal; `first_place` is fine).
- C is **case-sensitive** — `age`, `Age`, and `AGE` are three different names.
- You cannot use a **reserved word** as a name — words the language already uses for something else, like `int`, `float`, `return`, or `const` (more on `const` below).

Beyond what the compiler requires, good C code follows **conventions** — habits the whole C community shares so anyone's code is easy to read:

- Use **meaningful names**. `s` tells you nothing; `average_score` tells you exactly what's stored there. You are writing code for the next human who reads it — often that human is you, in six months.
- Use **snake_case** for variable names: all lowercase, words joined by underscores (`number_of_students`, **not** `numberOfStudents` or `NumberOfStudents`). This is the idiomatic style in C, even though other languages prefer different conventions.

Declaring, initializing, and re-assigning

Declaring a variable creates the box but doesn't necessarily put anything meaningful in it yet:

```
int score;
```

Initializing means giving it a value at the moment you declare it — almost always the better habit:

```
int score = 0;
```

You can also declare first and assign a value on a later line using `=`:

```
int score;
score = 0;
```

Once a variable already exists, using `=` again doesn't declare a new variable — it **re-assigns** (overwrites) the value already stored there:

```
int age = 16;
age = age + 1;    // reads the current value (16), adds 1, stores 17 back
```

Notice `age = age + 1;` is not an equation in the mathematical sense — `=` in C means "compute the right side, then store the result in the variable on the left," not "these two things are equal."

Why uninitialized variables are dangerous

Declaring a variable **without** initializing it leaves whatever bits happened to already be sitting at that spot in memory — a **garbage value**. It is not guaranteed to be `0`, or any predictable value at all:

```
int mystery;
printf("%d\n", mystery);    // could print anything -- don't do this
```

This is a classic source of bugs that are maddening to track down, because the program might *seem* to work (garbage value happens to be harmless) until it doesn't (garbage value happens to be huge, negative, or otherwise breaks your logic) on a different run or a different computer. The habit that avoids this entirely: **always initialize a variable when you declare it**, unless you are about to immediately read a value into it (as you'll do with `scanf` below).

const: values that should never change

Some values in a program should be fixed for its entire run — a mathematical constant, a maximum class size, a fixed price. Marking a variable `const` tells the compiler to enforce that:

```
const double PI = 3.14159;
const int MAX_STUDENTS = 30;
```

Try to assign a new value to a `const` variable anywhere later in the program, and `gcc` will refuse to compile it. That's a feature, not an annoyance — it turns "a value silently got changed somewhere it shouldn't have" from a confusing runtime bug into an immediate, obvious compiler error. Use `const` for any variable you can already tell should never change after it's set.

Reading user input with `scanf`

Every program until now has been a one-way conversation — it talks, you listen. `scanf` (from `stdio.h`, the same header that gives you `printf`) lets your program **read** input the user types.

```
int age;
printf("Enter your age: ");
scanf("%d", &age);
```

A few things to notice:

- The **format specifier** works the same way it did for `printf` in Module 2 — `%d` for `int`, `%f` for `float`, `%c` for a single `char`.
- The **& before the variable name** is required. It means "the address of this variable" — it tells `scanf` exactly *where* in memory to store the value it reads, rather than handing it a copy of whatever's currently there. You'll fully understand why this works the way it does in Module 7 (Pointers). For now, the rule is simple: **always write & before the variable name in `scanf`** (for the types you know so far — this changes once you meet strings and arrays in Module 5).
- Always `printf` a prompt *before* calling `scanf`, so the user knows the program is waiting for them to type something.

Reading a `float` works the same way:

```
float price;
printf("Enter a price: ");
scanf("%f", &price);
```

scanf with char — and the leading-space bug

Reading a single character looks like it should be just as simple:

```
char grade;
scanf("%c", &grade);
```

But there's a very common trap. Say your program first reads an `int` with `scanf("%d", &score)`, then immediately tries to read a `char`:

```
int score;
char grade;

scanf("%d", &score);
scanf("%c", &grade);    // BUG: this often doesn't wait for you to type
                        anything
```

When you type a number and press Enter, `scanf("%d", ...)` reads the digits but leaves the newline character (`\n`) from pressing Enter sitting in the input buffer, untouched. The very next `scanf("%c", ...)` immediately reads *that leftover newline* as if it were your answer — your program appears to skip right past asking for the character.

The fix is a single space, written before `%c`:

```
scanf(" %c", &grade);
```

That leading space tells `scanf`: "skip over any whitespace (spaces, tabs, or leftover newlines) sitting in the buffer first, then read the next real character." This one-character fix resolves one of the most common and confusing beginner bugs with `scanf` — if a `%c` read seems to get silently skipped, this is almost always why.

Putting it together: read, compute, print

Every interactive program you'll write follows the same three-step shape: **read input, compute something with it, print a result.**

```
float num1, num2;

printf("Enter the first number: ");
scanf("%f", &num1);

printf("Enter the second number: ");
scanf("%f", &num2);

float sum = num1 + num2;
printf("%.2f + %.2f = %.2f\n", num1, num2, sum);
```

This is exactly the pattern behind the mind-reader hook at the top of this module — read an age, compute a couple of values from it with arithmetic, print sentences that reference the real numbers the user typed. That's what makes it feel personal instead of generic.

Common beginner mistakes

- Forgetting `&` before a variable name in `scanf` — the program will usually crash or behave unpredictably.
- Forgetting the leading space before `%c` in `scanf(" %c", ...)` when a numeric read came before it.
- Using the wrong format specifier for the variable's type (e.g. `scanf("%d", &price)` where `price` is a `float`) — this silently reads garbage instead of warning you.
- Declaring a variable without initializing it and then reading its value before ever assigning one.
- Trying to re-assign a `const` variable after it's been set.
- Using non-descriptive names (`x`, `a`, `temp1`) instead of names that say what the value actually represents.

Try it yourself

1. Work through every file in **examples/**, compiling and running each one. Try feeding the `scanf` examples different input values.
2. Complete **exercises/exercise1.c**.
3. Build the **module project**.
4. Commit and push your work:

```
git add .
git commit -m "Complete Module 3: variables and input/output"
git push
```

Next: **Module 4 — Control Structures.**

Module 4 — Control Structures

Hook: Rock, Paper, Scissors vs. the computer

Compile and run this exactly as written — don't read ahead first:

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main(void) {
    srand(time(NULL));

    int playerChoice;
    printf("Rock (0), Paper (1), or Scissors (2)? ");
    scanf("%d", &playerChoice);

    int computerChoice = rand() % 3;

    printf("You chose:      ");
    if (playerChoice == 0) {
        printf("Rock\n");
    } else if (playerChoice == 1) {
        printf("Paper\n");
    } else {
        printf("Scissors\n");
    }

    printf("Computer chose: ");
    if (computerChoice == 0) {
        printf("Rock\n");
    } else if (computerChoice == 1) {
        printf("Paper\n");
    } else {
        printf("Scissors\n");
    }

    if (playerChoice == computerChoice) {
        printf("It's a tie!\n");
    } else if ((playerChoice == 0 && computerChoice == 2) ||
               (playerChoice == 1 && computerChoice == 0) ||
               (playerChoice == 2 && computerChoice == 1)) {
        printf("You win!\n");
    } else {
        printf("Computer wins!\n");
    }

    return 0;
}
```

```
gcc rps.c -o rps
./rps
```

Run it a few times and try all three choices. Two things in there are new: `if/else if/else`, which decide which lines of code actually run, and `rand()`, which gets you a random number. You've already met the building blocks these depend on — Module 2's `==`, `&&`, and `||` are exactly what's deciding the winner above. By the end of this module you'll understand every line of this, and you'll build a full Number Guessing Game using the same ideas in this module's project.

Why control structures?

Every program you've written so far runs top to bottom, once, no exceptions — line 1, then line 2, then line 3, done. That's fine for a fortune teller, but real programs need to **make decisions** ("if the guess is too high, say so") and **repeat work** ("keep asking until the player gets it right"). `if`, `switch`, `while`, `do-while`, and `for` are how C does both. They're called **control structures** because they control which statements run, and how many times.

if / else if / else

An `if` runs a block of code only when its condition is true. Recall from Module 2 that relational operators (`==`, `!=`, `<`, `>`, `<=`, `>=`) and logical operators (`&&`, `||`, `!`) produce 1 (true) or 0 (false) — that value is exactly what `if` checks.

```
int temperature = 28;

if (temperature >= 30) {
    printf("It's hot outside.\n");
} else if (temperature >= 20) {
    printf("It's a mild day.\n");
} else {
    printf("It's cold.\n");
}
```

C checks each condition **top to bottom** and runs the first block whose condition is true — then it skips every other branch, even if a later condition would also be true. `else if` lets you chain as many conditions as you need; the final `else` (optional) catches everything that didn't match any condition above it. Combine conditions with `&&` ("and", both must be true) and `||` ("or", at least one must be true) — see `examples/01_if_else.c`.

switch statements

When you're comparing **one variable** against several **exact values**, a `switch` is often cleaner than a long `if/else if` chain:

```
int dayNumber = 3;

switch (dayNumber) {
    case 1:
        printf("Monday\n");
        break;
    case 2:
        printf("Tuesday\n");
        break;
    case 3:
        printf("Wednesday\n");
        break;
    default:
        printf("Not a valid day number.\n");
        break;
}
```

`switch` jumps straight to the matching `case` and starts running code from there. `break` is what makes it **stop** and jump out of the switch — without it, execution keeps going into the next case too, whether it matches or not. This is called **falling through**, and forgetting `break` is one of the most common beginner switch bugs. See [examples/07_switch_fallthrough_demo.c](#) to watch it happen, then fix it. Always include a default case too — it's your safety net for values that don't match anything, the same job a final `else` does for `if`.

Note: `switch` can only compare against exact constant values — it can't do ranges or `&&/||` conditions. For those, you still need `if/else`.

while loops

A `while` loop repeats its body for as long as its condition stays true. The condition is checked **before** every run — including the first — so if it starts out false, the body never runs at all.

```
int countdown = 5;
while (countdown > 0) {
    printf("%d...\n", countdown);
    countdown = countdown - 1;
}
printf("Liftoff!\n");
```

Use a `while` loop when you're repeating **until something happens**, and you don't know in advance how many times that will take — like asking the player to guess a number until they get it right.

do-while loops

`do-while` is almost the same, except it checks its condition **after** running the body — so the body always runs **at least once**, no matter what the condition is:

```
int attempts = 10;
do {
    printf("This runs even though attempts is already 10.\n");
    attempts = attempts + 1;
} while (attempts < 3);
```

This matters anywhere you need to do something once before you can even check the condition — like asking for a player's first guess before you have anything to compare it to. See `examples/03_while_and_do_while.c` for a side-by-side comparison.

for loops

A `for` loop packs three things onto one line, separated by `::`

```
for (int i = 1; i <= 10; i = i + 1) {
    printf("%d\n", i);
}
```

1. **initialization** — `int i = 1;` runs once, before the loop starts.
2. **condition** — `i <= 10` is checked before every run; the loop stops the moment it's false.
3. **increment** — `i = i + 1` runs after every run of the body.

Use a `for` loop when you know **in advance** how many times you want to repeat something (print a multiplication table, roll a die 5 times). Use a `while` loop instead when the number of repetitions depends on something that happens *during* the loop. Anything you can write as a `for` loop, you could also write as a `while` loop with the initialization before it and the increment as the last line of the body — `for` just keeps those three pieces together so they're harder to forget.

break and continue

Both work inside any loop (`while`, `do-while`, or `for`):

- **break** immediately exits the loop entirely — no more checks, no more iterations.
- **continue** skips the *rest* of the current iteration only, then jumps straight to the next check of the condition — the loop keeps going.

```
for (int i = 1; i <= 15; i = i + 1) {
    if (i % 3 == 0) {
        continue; // skip multiples of 3, keep looping
    }
    if (i > 10) {
        break; // stop the loop completely once i passes 10
    }
    printf("%d\n", i);
}
```

Nesting control structures

Control structures can live inside each other — most commonly, an `if` inside a loop, to check something about each value as the loop produces it:

```
for (int i = 1; i <= 10; i = i + 1) {
    if (i % 2 == 0) {
        printf("%d is even\n", i);
    } else {
        printf("%d is odd\n", i);
    }
}
```

Each nested level is usually indented one more level than the one around it — that indentation is for humans (C ignores whitespace, as you saw in Module 1), but it's what makes nested code readable at a glance.

Random numbers with `rand()` and `srand()`

Computers can't generate truly random numbers — `rand()` (from `<stdlib.h>`) actually produces a long, fixed sequence of numbers that just *looks* random. That's good enough for games and simple programs like ours.

`srand()` "seeds" that sequence — without calling it, `rand()` gives you the exact same numbers every single time you run the program. Seeding with the current time, `srand(time(NULL))` (`time()` comes from `<time.h>`), makes the seed different each run, so the numbers actually change:

```
#include <stdlib.h>
#include <time.h>

srand(time(NULL)); // call this once, near the start of main
int roll = rand() % 6 + 1; // a number from 1 to 6
```

`rand()` alone returns a number between 0 and a very large maximum. `% 6` shrinks it down to 0-5, and `+ 1` shifts it up to 1-6. The general pattern for a random number between `min` and `max` (inclusive) is:

```
rand() % (max - min + 1) + min
```

See `examples/06_random_numbers.c`.

Common beginner mistakes

- Using `=` (assignment) instead of `==` (comparison) inside an `if` condition — `if (x = 5)` compiles, assigns 5 to `x`, and is always true, which is almost never what you meant.
- Forgetting `break` in a `switch` case, causing it to fall through into the next case.
- Writing a `while` loop whose condition never becomes false because you forgot to update the variable it depends on — an **infinite loop**. If your program seems frozen, this is almost always why (Ctrl+C stops it).
- Off-by-one errors in a `for` loop condition — `i < 10` runs 10 times (0-9), `i <= 10` runs 11 times (0-10). Decide which one you actually want and double check it.
- Calling `srand()` more than once, or inside a loop — call it **once**, near the top of `main`, before any calls to `rand()`.

- Confusing `break` and `continue` — `break` leaves the loop for good; `continue` just skips to the next round of it.

Try it yourself

1. Work through every file in **examples/**, compiling and running each one — including the intentionally broken `07_switch_fallthrough_demo.c`, so you see the bug before fixing it.
2. Complete **exercises/exercisel.c**.
3. Build the **module project**.
4. Commit and push your work:

```
git add .  
git commit -m "Complete Module 4: control structures"  
git push
```

Next: **Module 5 — Arrays & Strings.**

Module 5 — Arrays & Strings

Hook: crack the secret message

Compile and run this exactly as written — don't read ahead first:

```
#include <stdio.h>
#include <string.h>

int main(void) {
    char secret[] = "Duudbv krog pdqb ydoxhv lq d urz!";
    int shift = 3;
    int len = strlen(secret);

    for (int i = 0; i < len; i++) {
        char c = secret[i];
        if (c >= 'a' && c <= 'z') {
            secret[i] = 'a' + (c - 'a' - shift + 26) % 26;
        } else if (c >= 'A' && c <= 'Z') {
            secret[i] = 'A' + (c - 'A' - shift + 26) % 26;
        }
        // anything that isn't a letter (spaces, punctuation) is left alone
    }

    printf("Decoding secret message...\n");
    printf("%s\n", secret);

    return 0;
}
```

```
gcc hook.c -o hook
./hook
```

Gibberish goes in, an English sentence comes out. `secret` isn't a single value like the `ints` and `chars` you've used so far — it's a whole row of characters that the loop walks across one at a time, shifting each letter three places back through the alphabet. By the end of this module you'll understand exactly how `secret[i]` works, why `char secret[]` can hold a whole sentence, and you'll build your own encoder/decoder in the module project.

What is an array?

Every variable you've used so far holds exactly one value: one `int`, one `float`, one `char`. But plenty of real problems need a *bunch* of related values — five quiz scores, thirty students' ages, a whole word. Declaring 30 separate variables (`score1`, `score2`, `score3`, ...) would be miserable to write and impossible to loop over.

An **array** is a fixed number of values of the same type, stored back to back in memory under one name:

```
int scores[5]; // 5 ints, not set yet
int prices[5] = {90, 85, 77, 60, 100}; // 5 ints, set immediately
```

`scores[5]` reserves 5 slots. `prices[5] = {...}` reserves 5 slots *and* fills them in, left to right, in the order you list them.

Indexing starts at 0

Each slot in an array has a position number called an **index**, and in C indexing always starts at **0**, not 1. For an array of size 5, the valid indexes are 0, 1, 2, 3, 4 — the last valid index is always `size - 1`, never `size`.

```
printf("%d\n", prices[0]); // 90  <- the FIRST element
printf("%d\n", prices[4]); // 100 <- the LAST element
```

This trips up almost every beginner at least once: reaching for `prices[5]` expecting "the 5th price" and getting the price that doesn't exist. `prices[5]` would be the **6th** slot of a 5-slot array — one past the end. More on why that's dangerous below.

Iterating with a for loop

A for loop is the natural match for "do something to every element":

```
for (int i = 0; i < 5; i++) {
    printf("prices[%d] = %d\n", i, prices[i]);
}
```

`i` takes the values 0, 1, 2, 3, 4 — exactly the array's valid indexes — and stops before `i` ever reaches 5. This pattern (`i = 0, i < size, i++`) is the standard way to visit every element of an array, and you'll use it constantly: summing values, finding a maximum, searching for something, printing everything out.

Array bounds: C will not save you

Here's the single most important safety lesson in this module: **C does not check array bounds for you**. Many languages throw a clear, helpful error the instant you try to read or write past the end of an array. C does not. If you write `scores[5]` on a 5-element array, the compiler happily generates code for it — code that reads or writes whatever memory happens to sit right after your array. This is called **undefined behavior**, and the word "undefined" is doing a lot of work: it might print a weird number, it might silently corrupt some *other* variable elsewhere in your program, it might work fine today and crash tomorrow, it might crash immediately. There is no guarantee, and no helpful error message — the bug can show up far away from its actual cause, which makes it one of the hardest kinds of bug to track down.

The discipline that prevents this: **always know your array's real size, and never trust an index blindly**.

```
int size = 5;
int index = get_some_index(); // from input, a calculation, wherever

if (index >= 0 && index < size) {
    printf("%d\n", scores[index]);
} else {
    printf("Index %d is out of bounds.\n", index);
}
```

The classic way this bites beginners is an off-by-one loop condition: writing `i <= size` instead of `i < size` walks one slot too far on the very last pass. Whenever a loop touches an array, double check the condition against the array's actual size.

C-strings: arrays of characters

You already know `char grade = 'A';` — a single character, single quotes. A **string** in C is different: it's an *array* of `char`, written with double quotes, and printed with `%s` instead of `%c`:

```
char grade = 'A';           // ONE character
char name[20] = "Ama";      // a STRING — an array that can hold up to 19
                             // real characters (see why below)
printf("Grade: %c\n", grade);
printf("Name: %s\n", name);
```

Here's the part that catches everyone at first: C secretly stores one extra, invisible character right after "Ama" — `'\0'`, the **null terminator**. It marks "the real string data ends here." So `name[20]` actually looks like this in memory:

```
name[0]='A'  name[1]='m'  name[2]='a'  name[3]='\0'  name[4..19]=unused
```

`printf("%s", ...)`, `strlen`, and every other string function scan forward until they hit that `'\0'` and stop there — they have no other way of knowing where your string ends. This is also why a `char name[20]` can only ever hold a string of **19** real characters, not 20: one slot is always reserved for the terminator. Forget to leave room for it (or overwrite it), and every string function that touches that array walks straight past your data into undefined behavior.

Reading a line of text safely

Module 3 used `scanf("%d", ...)` and `scanf("%f", ...)` for numbers. Reading text with `scanf("%s", ...)` has two serious problems: it stops at the first space (so "Kofi Mensah" becomes just "Kofi"), and it has no idea how big your array is — a long enough input overflows the buffer, which is exactly the out-of-bounds danger from above.

`fgets` fixes both:

```
char name[20];
printf("What is your name? ");
fgets(name, sizeof(name), stdin);
```

`sizeof(name)` tells `fgets` exactly how many bytes it's allowed to write, so it can never overflow the array no matter what the user types. The trade-off: `fgets` keeps the newline character from the Enter key at the end of the string. It's usually not wanted, so it's common to strip it off:

```
name[strcspn(name, "\n")] = '\0';
```

`strcspn(name, "\n")` finds the index of the first `'\n'` in `name`; writing `'\0'` there shortens the string right at that point.

Useful `<string.h>` functions

Function	What it does
<code>strlen(s)</code>	Number of real characters in <code>s</code> (not counting <code>'\0'</code>)
<code>strcpy(dest, src)</code>	Copies <code>src</code> into <code>dest</code> (<code>dest</code> must be big enough)
<code>strcmp(a, b)</code>	Returns 0 if <code>a</code> and <code>b</code> are exactly equal
<code>strcat(dest, src)</code>	Appends <code>src</code> onto the end of <code>dest</code>

```
char city[20] = "Kumasi";
printf("%zu\n", strlen(city));           // 6

char copy[20];
strcpy(copy, city);                     // copy now holds "Kumasi"

if (strcmp(copy, city) == 0) {           // true - same contents
    printf("They match!\n");
}

char greeting[40] = "Welcome to ";
strcat(greeting, city);                 // greeting is now "Welcome to Kumasi"
```

Notice `strcmp`, not `==`. Comparing two strings with `==` compares where they live in memory, not what characters they contain — it will almost never do what you want. Always reach for `strcmp` instead.

A preview: 2D arrays

Some problems need a grid instead of a single row — a tic-tac-toe board, a seating chart, a spreadsheet of numbers. C lets you declare a **2D array** for exactly that:

```
int grid[3][3]; // 3 rows, each holding 3 ints
```

You won't need these yet — plain 1D arrays are enough for this module — but it's worth knowing the idea exists for when a problem needs more than one dimension.

One more preview, just so the name doesn't surprise you later: an array name like `scores` is closely related to something called a **pointer** — you'll see exactly what that means and how it works in Module 7.

Common beginner mistakes

- Using index `size` instead of `size - 1` for the last element (off-by- one), or looping with `i <= size` instead of `i < size`.
- Reading or writing an index outside the array's bounds — C will not warn you; the bug can show up somewhere completely unrelated later on.

- Declaring a `char` array too small for the string plus its `'\0'` (e.g. `char name[3] = "Ama";` — that needs at least 4 slots, not 3).
- Using `scanf("%s", ...)` for user input instead of `fgets` — it splits on whitespace and has no protection against buffer overflow.
- Comparing strings with `==` instead of `strcmp`.
- Forgetting that `fgets` keeps the trailing newline, and being confused when a "matching" string comparison mysteriously fails.

Try it yourself

1. Work through every file in **examples/**, compiling and running each one.
2. Complete **exercises/exercisel.c**.
3. Build the **module project**.
4. Commit and push your work:

```
git add .  
git commit -m "Complete Module 5: arrays and strings"  
git push
```

Next: **Module 6 — Functions.**

Module 6 — Functions

Hook: the same receipt, two very different mains

Compile and run both of these — don't read ahead first.

Version A:

```
#include <stdio.h>

int main(void) {
    double price1 = 4.50;
    int qty1 = 3;
    double subtotal1 = price1 * qty1;
    double tax1 = subtotal1 * 0.15;
    double total1 = subtotal1 + tax1;
    printf("Coffee: %d x $%.2f = $%.2f (tax $%.2f) => total $%.2f\n",
           qty1, price1, subtotal1, tax1, total1);

    double price2 = 2.25;
    int qty2 = 5;
    double subtotal2 = price2 * qty2;
    double tax2 = subtotal2 * 0.15;
    double total2 = subtotal2 + tax2;
    printf("Muffin: %d x $%.2f = $%.2f (tax $%.2f) => total $%.2f\n",
           qty2, price2, subtotal2, tax2, total2);

    return 0;
}
```

Version B:

```
#include <stdio.h>

void printReceiptLine(char itemName[], double unitPrice, int quantity) {
    double subtotal = unitPrice * quantity;
    double tax = subtotal * 0.15;
    double total = subtotal + tax;
    printf("%s: %d x $%.2f = $%.2f (tax $%.2f) => total $%.2f\n",
           itemName, quantity, unitPrice, subtotal, tax, total);
}

int main(void) {
    printReceiptLine("Coffee", 4.50, 3);
    printReceiptLine("Muffin", 2.25, 5);
    return 0;
}
```

```
gcc versionA.c -o versionA && ./versionA
gcc versionB.c -o versionB && ./versionB
```

Same numbers in, same receipt lines out — byte for byte. Version B is less than half the length, and if the tax rate ever changes, you fix it in exactly **one** place instead of hunting down every copy-pasted block. That's the entire pitch for this module: a **function** is a named, reusable chunk of code you write

once and use as many times as you like. By the end of this module you'll refactor a much bigger program the same way. You can see this exact comparison, with a bit more detail, in `examples/01_why_functions_before_and_after.c`.

Why functions?

Think back to your Module 4 Number Guessing Game project: the computer picks a random number, and `main()` loops — reading a guess, checking whether it's in range, comparing it to the secret number, and printing "too high," "too low," or "correct." For a first version, all of that living inside one `main()` is fine.

But picture growing that game a little: add a limit on the number of guesses, add a "play again?" option, add a difficulty setting that changes the range. Every new feature means `main()` gets a few lines longer and a bit more tangled — the input-reading code, the validation code, the comparison code, and the printing code all interleaved. Six months from now (or six minutes from now, debugging it), it's hard to answer a simple question like "where exactly do we decide the player won?" because that logic is mixed in with everything else.

Functions fix this by giving each *job* a name and a boundary. "Get a valid guess from the player" becomes a function. "Check the guess" becomes a function. `main()` shrinks down to reading like a to-do list: get a guess, check it, print the result, repeat. You'll do exactly this refactor to your own guessing game in this module's project.

Breaking a program into functions gives you three things at once:

- **Reusability** — write the logic once, call it from anywhere, as many times as you like (that's the receipt hook, above).
- **Readability** — a well-named function tells you what a block of code does without you having to read its body. `checkGuess(guess, secret)` is self-explanatory; twelve lines of `if/else` inline in `main()` aren't, at a glance.
- **Testability** — you can try a function on its own, with known inputs, and check it gives the right output, without running your whole program end to end.

Declaring and calling a function

A function has three parts: a **header** (what it's called, what it takes in, what it gives back), a **body** (the code that runs), and, separately, every place you **call** it.

```
void printStarRow(int count) {
    for (int i = 0; i < count; i++) {
        printf("*");
    }
    printf("\n");
}

int main(void) {
    printStarRow(3);
    printStarRow(8);
    return 0;
}
```

Reading the header left to right: `void` is the **return type** (this function doesn't send any value back — more on that soon), `printStarRow` is the **name**, and `(int count)` is the **parameter list** — the inputs this function expects. Every time you write `printStarRow(3) ;`, you're **calling** the function, and 3 flows into `count` for that one run.

See `examples/02_declaring_and_calling.c`.

Prototypes: telling the compiler what's coming

C reads your file from top to bottom, once. If `main()` calls a function that hasn't been defined yet — because its full definition is further down the file — the compiler has no idea it exists and refuses to compile.

A **prototype** (also called a **declaration**) solves this. It's the function's header line, on its own, ending in a semicolon, with no body:

```
void describeTemperature(int celsius); // prototype

int main(void) {
    describeTemperature(35); // compiler already knows this exists
    return 0;
}

void describeTemperature(int celsius) { // full definition, further down
    if (celsius >= 30) {
        printf("%d C: that's hot.\n", celsius);
    } else {
        printf("%d C: that's not so hot.\n", celsius);
    }
}
```

The prototype is a promise: "a function with this exact name, these parameter types, and this return type exists somewhere in this file." The compiler holds you to it — if the real definition doesn't match, or never shows up, you'll get an error at link time. This is exactly why you'll often see a block of prototypes near the top of a C file, above `main()`: it lets you write `main()` first, reading top-to-bottom like a summary of the program, with the messy implementation details below it.

See `examples/03_prototypes.c`.

Parameters, arguments, and return values

These two words get mixed up constantly, so pin them down now:

- A **parameter** is the named placeholder in the function's own header — it's part of how the function is *written*.
- An **argument** is the actual value you hand over at the *call site*.

```
int add(int a, int b) { // a and b are PARAMETERS
    return a + b;
}

int total = add(3, 4); // 3 and 4 are ARGUMENTS
```

A function can send a result back to its caller with `return`. The type before the function's name promises what type that will be:

```
int isEven(int number) {
    if (number % 2 == 0) {
        return 1;
    } else {
        return 0;
    }
}
```

`return` immediately exits the function with that value — any code after a `return` in the same path never runs. You can use the returned value directly (`printf("%d\n", isEven(7));`) or store it in a variable first.

See `examples/04_parameters_and_return_values.c`.

void functions

Not every function needs to hand a value back. A function whose job is just to *do* something — print a banner, update the screen, play a sound — declares `void` as its return type, meaning "nothing comes back." You already met one above: `printStarRow`. A `void` function can still use a bare `return`; (no value) to exit early, but it's optional.

Scope: where a variable lives

A variable declared inside a function only exists **inside that function**, for as long as that function is running. This is called its **scope**. Once the function returns, its local variables are gone — nothing outside can see them, read them, or accidentally clash with a variable of the same name in a different function.

```
void printWelcomeBanner(char userName[]) {
    int line = 1;           // exists only inside this function
    printf("%d) Welcome, %s!\n", line, userName);
}

int main(void) {
    printWelcomeBanner("Ama");
    // printf("%d\n", line); // ERROR: `line` doesn't exist out here
    return 0;
}
```

A **global variable** — declared outside every function — is the opposite: every function can read and change it. It compiles, and sometimes you'll see it in other people's code, but avoid it as a beginner. When any function anywhere can silently change a shared variable, tracking down *which* function caused a wrong value becomes a real headache as programs grow. Passing values in as parameters and getting results back with `return` keeps each function's effects visible and contained.

See `examples/05_void_functions_and_scope.c`.

Passing an array to a function

You can pass an array to a function just like any other value — you just also pass its size as a separate argument, since C doesn't track an array's length for you once it's inside the function:

```
int findMax(int arr[], int size) {
    int max = arr[0];
    for (int i = 1; i < size; i++) {
        if (arr[i] > max) {
            max = arr[i];
        }
    }
    return max;
}
```

Here's the part that's genuinely different from every other type you've passed to a function so far: **when you pass an array, the function can modify the original array back in `main()`** — not just its own private copy. An `int` or `double` argument gets copied in, and changes inside the function stay inside the function. Arrays don't work that way. You'll find out exactly why in Module 7 — for now, just know arrays are the exception, and treat any function that takes an array as a function that might change it.

See `examples/06_passing_arrays_to_functions.c`.

Refactoring: turning a mess into functions

Refactoring means changing how code is organized without changing what it does. It's the single most practical skill in this module. The process is always the same:

1. Find a self-contained piece of logic — something you could describe in one short sentence ("this part checks if the guess is too high or too low").
2. Give that sentence a function name (`checkGuess`).
3. Move the code into a new function with that name, turning the values it depended on into parameters, and whatever it produced into a `return` value (or a `printf`, if its whole job is printing).
4. Replace the original code with a call to your new function.
5. **Recompile and test immediately.** Refactoring should never change behavior — if the output changes, you introduced a bug, not an improvement.

That's exactly what you did conceptually in the hook at the top of this module (turning duplicated receipt math into `printReceiptLine`), and it's exactly what this module's project asks you to do to your whole Number Guessing Game.

Common beginner mistakes

- Forgetting a function's **prototype** when it's defined below `main()` — the compiler will complain about an undeclared or implicitly declared function, often pointing at the *call*, not the missing prototype.
- Mixing up **parameters** and **arguments** in conversation — harmless day to day, but know the difference when reading error messages.

- Writing `void` on a function and then trying to use its result: `int x = printStarRow(3);` won't compile — a `void` function has no value to give back.
- Forgetting `return` in a non-`void` function, or returning the wrong type. The compiler will often warn you, but always double check.
- Trying to use a local variable outside the function it was declared in — remember, its scope ends at the function's closing `}`.
- Forgetting to pass an array's size as its own parameter, or assuming a function won't change the array you passed it, when it does.

Try it yourself

1. Work through every file in **examples/**, compiling and running each one.
2. Complete **exercises/exercise1.c**.
3. Build the **module project**.
4. Commit and push your work:

```
git add .  
git commit -m "Complete Module 6: functions"  
git push
```

Next: **Module 7 — Pointers.**

Module 7 — Pointers

Hook: the magic swap that isn't

Compile and run this exactly as written — don't read ahead first:

```
#include <stdio.h>

void swapBroken(int a, int b) {
    int temp = a;
    a = b;
    b = temp;
    printf("    inside swapBroken: a = %d, b = %d\n", a, b);
}

int main(void) {
    int x = 3;
    int y = 7;

    printf("Before: x = %d, y = %d\n", x, y);
    swapBroken(x, y);
    printf("After:  x = %d, y = %d\n", x, y);

    return 0;
}
```

```
gcc swap.c -o swap
./swap
```

Look closely at the output. Inside `swapBroken`, `a` and `b` swap just fine — but back in `main`, `x` and `y` are completely unchanged. The function did exactly what you told it to. So why didn't it work?

The answer is coming in this module, and once you have it, you'll fix this with a real, working swap using nothing but the tools below. This is also the exact question that's been waiting since Module 6: why could a function change the contents of an array you passed it, but never change a plain `int`, `float`, or `char` the same way? Same answer.

What is a memory address, really?

Every variable you create has to physically live somewhere while your program runs — in **RAM (random-access memory)**. You can think of RAM as an enormous row of numbered mailboxes; when you write `int age = 25;`, the compiler picks a free mailbox, labels it `age` (just for your convenience while reading the code), and stores 25 inside it. The *number* of that mailbox is called its **address**.

You've already been using addresses without necessarily thinking of them that way — every time you wrote `scanf("%d", &age);` back in Module 3, that `&` meant "don't give `scanf` the value of `age`, give it the *address* of `age`, so `scanf` can reach in and store what the user typed directly into that mailbox." That's the whole mystery behind `&` finally explained.

You can print any variable's address yourself, using `&` and the `%p` format specifier:

```
int age = 25;
printf("age lives at address: %p\n", (void *)&age);
```

The address itself will look like a strange string of hex digits (e.g. `0x7ffee3a1c9ac`). You will never need to memorize or predict what it says — what matters is that it exists, and that a **pointer** is simply a variable built to store one.

Declaring a pointer

A pointer is a variable, just like any other — except instead of holding a number or a character, it holds an *address*. You declare one with a `*` between the type and the name:

```
int *p;
```

Read this as "p is a pointer to `int`" — meaning: whatever address `p` holds, it promises that address contains an `int`. This matters, because the computer needs to know how many bytes to read starting from that address; a pointer to `int` and a pointer to `char` read different amounts of memory when you use them.

The dual meaning of `*` — the single biggest beginner trap

Here is the part that trips up almost everyone at first: the `*` symbol means **two completely different things** depending on where it shows up, and C gives you no visual hint about which one you're looking at.

```
int *p = &age;    // (1) declaration: * means "p is a pointer"
*p = 100;         // (2) dereference: * means "the value at the address p
holds"
```

- In line (1), `*` is part of *declaring* `p`'s type. It only appears once, at the moment you introduce the variable.
- In line (2), `*` is the **dereference operator**. Applied to a pointer that already exists, `*p` means "go to the address stored in `p`, and give me (or let me set) the value that lives there."

The rule of thumb: if you're introducing a brand-new variable name right after a type, `*` is part of the declaration. Anywhere else, `*` is asking to follow the pointer to what it points to.

Assigning and dereferencing a pointer

Putting declaration and dereference together:

```
int age = 25;
int *p = &age;    // p now holds the ADDRESS of age -- "p points to age"

printf("%d\n", *p); // dereference: "the value at the address p holds" -> 25
```

`p` and `age` are two different variables. `p` doesn't contain 25 — it contains *the address where 25 lives*. `*p` is how you ask C to follow that address and hand you what's actually stored there.

Modifying a value through a pointer

Because `*p` means "the value at the address `p` holds," you can also *assign* through it:

```
*p = 100;    // go to the address p holds, and store 100 there
printf("%d\n", age); // prints 100 -- age itself changed!
```

`p` never changed — it still points to the same address. What changed is the value **sitting at** that address, which happens to be `age`. This is the mechanism that makes pointers useful: they let one part of your program reach out and change a variable that belongs to another part.

Pass-by-value vs. pass-by-pointer: solving the hook

Back to `swapBroken`. When you call `swapBroken(x, y)`, C copies the *values* of `x` and `y` into brand-new local variables `a` and `b`. Swapping `a` and `b` swaps two copies that live nowhere near `x` and `y` — and disappear the moment the function returns. This is called **pass-by-value**, and it's how *every* plain parameter (`int`, `float`, `char`, ...) works in C, always.

To actually reach `x` and `y`, you must give the function their *addresses* instead — pass-by-pointer:

```
void swapWorks(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

// called as:
swapWorks(&x, &y);
```

Now `a` holds the address of `x`, and `b` holds the address of `y`. `*a` and `*b` reach all the way back into `main`'s variables and swap the real thing. Compile and run this version — `x` and `y` really do swap this time.

C has no separate "pass by reference" keyword the way some other languages do. **Pointers are how C does it** — explicitly, by you writing `&` at the call site and `*` inside the function.

Arrays and pointers: Module 6's mystery, solved

In Module 6, you learned that a function *can* modify the caller's array directly — unlike a plain `int`. Now you know why: when you pass an array to a function, it **decays into a pointer to its first element**. You were never passing a copy of the whole array — you were passing an address, exactly like `&x` above, whether you wrote `&` or not.

That means these two function signatures are interchangeable:

```
int findMax(int arr[], int size) { ... } // Module 6's version
int findMax(int *arr, int size) { ... } // exactly the same function
```

Both receive the address of the array's first element. `arr[i]` still works either way — it's just friendlier notation for "the value `i` slots past the address `arr` holds."

NULL pointers

A pointer that isn't pointing at a real variable yet should be set to `NULL` — a special value meaning "this points to nothing, on purpose."

```
int *p = NULL;

if (p != NULL) {
    printf("%d\n", *p);
} else {
    printf("p isn't pointing anywhere yet.\n");
}
```

Always check for `NULL` before dereferencing a pointer you're not certain has been pointed at something. Dereferencing `NULL` — or any uninitialized "garbage" pointer that was never assigned an address at all — is undefined behavior. Most often it crashes your program immediately (a "segmentation fault"); sometimes, worse, it silently reads or corrupts memory that isn't yours and doesn't crash at all.

Common beginner mistakes

- Forgetting `&` when a function needs an address (e.g. `swapWorks(x, y)` instead of `swapWorks(&x, &y)`) — this is usually a compiler warning about mismatched types, not silence, so read those warnings.
- Confusing declaration-`*` with dereference-`*` — remember: only at the moment a pointer variable is introduced does `*` mean "this is a pointer type."
- Dereferencing a pointer that was never assigned an address (garbage) or was set to `NULL` — always check `if (p != NULL)` first when you're not certain.
- Assuming a pointer and the variable it points to are "the same variable" — they're two separate variables; one just happens to store the other's address.
- A sharp edge for later: `*p++` does NOT mean what it looks like at a glance — `++` binds tighter than `*`, so precedence rules matter a lot once you start combining pointer arithmetic with increment/decrement. You don't need to master this now; just know it's coming, and use parentheses like `(*p)++` when in doubt.

Try it yourself

1. Work through every file in **examples/**, compiling and running each one.
2. Complete **exercises/exercisel.c**.
3. Build the **module project**.
4. Commit and push your work:

```
git add .
git commit -m "Complete Module 7: pointers"
git push
```

Next: **Module 8 — Structs & File I/O.**

Module 8 — Structs & File I/O

Hook: a program that remembers

Compile and run this exactly as written — don't read ahead first:

```
#include <stdio.h>

typedef struct {
    int highScore;
} GameData;

int main(void) {
    GameData data;
    FILE *file = fopen("save.txt", "r");

    if (file == NULL) {
        printf("No save file found -- starting fresh.\n");
        data.highScore = 0;
    } else {
        fscanf(file, "%d", &data.highScore);
        fclose(file);
        printf("Welcome back! Your previous high score was %d.\n",
data.highScore);
    }

    int newScore;
    printf("Enter your score this round: ");
    scanf("%d", &newScore);

    if (newScore > data.highScore) {
        data.highScore = newScore;
        printf("New high score!\n");
    }

    file = fopen("save.txt", "w");
    if (file == NULL) {
        printf("Error: could not save your score.\n");
        return 1;
    }
    fprintf(file, "%d", data.highScore);
    fclose(file);

    printf("High score (%d) saved. Run this program again and see for
yourself.\n", data.highScore);
    return 0;
}
```

```
gcc hook.c -o hook
./hook
```

Enter a score, let it tell you it saved a new high score, then **run `./hook` again**. This time it greets you by name of achievement — "Welcome back! Your previous high score was..." — using data from a run of the program that already ended. Every variable from that first run is long gone from memory; only the

file survived. By the end of this module you'll understand every line of this, and you'll build a bigger version of the same trick in the module project.

Grouping data that belongs together

Modules 1–7 gave you variables, arrays, and functions, but every array you've built so far holds only ONE kind of thing: an array of `int`, an array of `char`. Real data doesn't come in single columns — a student has a name AND an age AND a GPA, all at once, all describing the same person.

Without structs, you'd track that with **parallel arrays**:

```
char names[3][30] = {"Ama", "Kwame", "Efua"};
int ages[3]       = {20, 22, 21};
float gpas[3]     = {3.8f, 3.2f, 3.9f};
```

This works, but it's fragile: `names[i]`, `ages[i]`, and `gpas[i]` only describe "the same student" because you remember to keep all three arrays in sync by hand. Sort one and forget the other two, and Ama's name silently ends up next to Kwame's age. The compiler has no idea these three arrays are related, so it can't warn you when they drift apart.

Defining a struct

A **struct** groups related variables — called **members** — into a single new type:

```
struct Student {
    char name[30];
    int age;
    float gpa;
};
```

This doesn't create a variable yet — it defines a new *type* called `struct Student`, the same way `int` or `char` is a type. To actually get a variable, declare one like anything else:

```
struct Student s1;
```

Accessing members with .

Use the **dot operator** (`.`) to read or set a member of a struct variable:

```
strcpy(s1.name, "Ama"); // char arrays still need strcpy (Module 5)
s1.age = 20;
s1.gpa = 3.8f;

printf("%s is %d\n", s1.name, s1.age);
```

You can also fill in every member at once, in declaration order:

```
struct Student s2 = {"Kwame", 22, 3.2f};
```

typedef: a shorter name for the same type

Typing `struct Student` every time you declare a variable gets old fast. `typedef` lets you attach a shorter alias to the type:

```
typedef struct {
    char name[30];
    int age;
    float gpa;
} Student;

Student s1;    // instead of: struct Student s1;
```

This is **purely a naming convenience** — `Student` and the `struct` it names are exactly the same type underneath, using exactly the same amount of memory, with exactly the same members. `typedef` just saves you keystrokes and reads a little cleaner. From here on, this module uses the `typedef` style.

Arrays of structs

The whole point of fixing the parallel-arrays problem: one array, where each slot is a *complete* student.

```
Student class[3] = {
    {"Ama", 20, 3.8f},
    {"Kwame", 22, 3.2f},
    {"Efua", 21, 3.9f}
};

for (int i = 0; i < 3; i++) {
    printf("%s is %d, GPA %.1f\n", class[i].name, class[i].age, class[i].gpa);
}
```

Looping looks exactly like the array loops from Module 5 — you're just reaching one member deeper with `.` on each element.

Structs and pointers: the arrow operator

Module 7 taught you `&` (address-of) and `*` (dereference). Both still work exactly the same way on structs:

```
Student s1 = {"Ama", 20, 3.8f};
Student *p = &s1;

printf("%d\n", (*p).age);    // dereference p, THEN access .age
```

The parentheses around `*p` are required — `.` binds tighter than `*`, so `*p.age` would (incorrectly) try to do `p.age` first. Because `(*p).member` is so common, C gives you a dedicated operator for it: the **arrow operator**, `->`.

```
printf("%d\n", p->age);    // exactly the same thing as (*p).age
```

`p->age` and `(*p).age` mean *identically* the same thing — always prefer `->`, it's what every C programmer expects to read. And because `p` *points at* `s1` rather than holding a separate copy, writing through the pointer really does change the original:

```
p->age = 21;
printf("%d\n", s1.age); // prints 21 -- same variable
```

Passing structs to functions by pointer

Module 7's biggest lesson was: passing a variable to a function normally passes a *copy*, so changes inside the function don't escape it — unless you pass a pointer instead. That rule applies to structs exactly as it applies to `int`:

```
void birthdayByValue(Student s) {
    s.age = s.age + 1; // only changes the LOCAL copy
}

void birthdayByPointer(Student *s) {
    s->age = s->age + 1; // changes the CALLER's struct
}

int main(void) {
    Student s1 = {"Ama", 20, 3.8f};
    birthdayByValue(s1);
    printf("%d\n", s1.age); // still 20

    birthdayByPointer(&s1);
    printf("%d\n", s1.age); // now 21
}
```

Rule of thumb: pass a struct **by pointer** whenever a function needs to *modify* the caller's copy — `&s1` at the call site, `Student *s` as the parameter, `s->member` inside the function.

Basic file I/O

Every program you've written so far forgets everything the moment it exits — all its variables live in memory, and memory is wiped clean when the process ends. To make data survive, you have to write it to a **file** on disk.

Opening a file: `fopen`

```
FILE *file = fopen("students.txt", "w");
```

`fopen` takes a filename and a **mode** string describing what you intend to do:

Mode	Meaning
"w"	Write. Creates the file if missing; erases existing contents if it already exists.
"r"	Read. Fails if the file doesn't exist.

Mode	Meaning
"a"	Append. Like "w", but adds to the <i>end</i> of an existing file instead of erasing it.

Always check for *NULL*

`fopen` can fail — the file doesn't exist (in "r" mode), a bad path, no permission to write there. When it fails, it returns `NULL` instead of a usable `FILE*`. **Always** check before using the result:

```
if (file == NULL) {
    printf("Error: could not open the file.\n");
    return 1;
}
```

Skipping this check is one of the most common real-world C bugs: the program crashes the instant it tries to read or write through a `NULL` file pointer, often with a confusing error far from the real cause.

Writing: *fprintf*

Works exactly like `printf`, except the first argument says *where* the text goes:

```
fprintf(file, "%s %d %.2f\n", s.name, s.age, s.gpa);
```

Reading back: *fscanf*

Works exactly like `scanf` (Module 3), except it reads from a file instead of the keyboard. It returns how many values it successfully matched, which is exactly what you need to loop until the file runs out:

```
Student s;
while (fscanf(file, "%29s %d %f", s.name, &s.age, &s.gpa) == 3) {
    // got one full student -- store it, e.g. class[count++] = s;
}
```

Closing: *fclose* — and why it matters

```
fclose(file);
```

Text you write with `fprintf` isn't necessarily on disk the instant you call it — it can sit in a memory buffer for efficiency, and only actually get written when the buffer fills up, the program ends normally, or you close the file. `fclose` forces that write to happen *now*, and releases the file so nothing else is blocked from using it. A program that writes data but crashes (or is killed) before closing its files can lose data it thinks it already saved. Get in the habit: every successful `fopen` gets a matching `fclose`.

Putting it together

The hook at the top of this module is the whole pattern in miniature: write struct data to a file with `fprintf` in one run, read it back with `fscanf` into a fresh array in a completely separate run, and the data behaves as if the program never forgot it. **examples/06_writing_a_file.c** and

`examples/07_reading_a_file.c` show this with a full array of structs instead of a single number — run `06` first, then `07`, and watch the array come back exactly as you saved it.

This is also exactly what **Module 9's capstone project** requires: a struct type stored in an array, saved and reloaded so the program's data survives being closed and reopened. Everything in this module exists to make that requirement feel routine by the time you get there.

Common beginner mistakes

- Forgetting to check `fopen`'s return value for `NULL` before using it — this crashes the program the moment the file doesn't exist or can't be opened.
- Forgetting `fclose` — your data may never actually make it to disk, or later attempts to open the same file may behave unexpectedly.
- Opening a file in `"w"` mode when you meant `"a"` (or vice versa) and accidentally erasing everything that was there.
- Writing `*p.age` instead of `(*p).age` or `p->age` — `.` binds tighter than `*`, so this doesn't do what it looks like it does.
- Mismatching the `fscanf` format string with what `fprintf` actually wrote (wrong order, wrong number of fields, or a name containing a space when `%s` stops at the first space) — read the file back with a text editor if a read isn't behaving as expected.
- Forgetting the size limit in `%29s`-style reads into a fixed-size `char` array, allowing a read to overflow the array (Module 5's string-safety lesson still applies here).

Try it yourself

1. Work through every file in `examples/`, compiling and running each one. Run `06_writing_a_file.c`, then `07_reading_a_file.c` right after it, in the same folder — you're proving to yourself that the second run really does read what the first run wrote.
2. Complete `exercises/exercise1.c`.
3. Build the **module project** — run it, save, quit, and run it again to prove your data survived.
4. Commit and push your work:

```
git add .
git commit -m "Complete Module 8: structs and file I/O"
git push
```

Next: **Module 9 — Capstone Project.**

Module 9 — Capstone Project

This is where every module comes together: variables, control structures, arrays, strings, functions, pointers, structs, and file I/O, combined into one real program you'll be proud to link from your GitHub profile.

1. How teams are formed

Your instructor announces the class size before this module starts, which sets your group size:

Class size	Group size	Ratio
Up to 20 students	Pairs	2:1
More than 20 students	Groups of 4	4:1
Approved solo ("daring") students, any class size	Individual	1:1

Going solo ("daring" track): any student may request to build the capstone alone instead of in a group. This isn't a smaller workload — it's the same core requirements (§3) **plus every stretch goal** listed for your chosen project (§4), since you don't have teammates splitting the work. Request this from your instructor *before* Sprint 0 (§5) so GitHub Classroom can set up your repo correctly. Going solo is a good fit if you're already comfortable with everything through Module 8 and want the extra stretch — it is not a reward for wanting to avoid teammates.

2. Forming your team in GitHub Classroom

1. Your instructor posts **one Group Assignment invite link** for the capstone.
2. Open it, sign in with GitHub, and either **create a new team** (name it after your project idea, e.g. `library-system-team1`) or **join an existing team** a teammate already created. Team size is enforced by the size from the table above — Classroom won't let you overfill it.
3. Solo/"daring" students accept the same link but create a team of one.
4. GitHub Classroom provisions **one shared repository** for your team, copied from the capstone starter template. Every teammate clones the *same* repo:

```
git clone https://github.com/YOUR-CLASSROOM-ORG/capstone-yourteamname.git
cd capstone-yourteamname
```

1. Everyone commits under their **own** Git identity (`git config user.name/user.email` from Module 0) — this is how your instructor sees who contributed what. A repo with every commit from one teammate and none from the other is a problem worth fixing early, not at the deadline.

3. Requirements for every project (any topic, any team size)

Regardless of which idea you pick from §4, your submission must include:

- **At least one `struct` type**, used in an array or similar collection (Module 8) — this is your program's core data.
- **File I/O** (Module 8): your program's data must survive being closed and reopened. If you quit and relaunch, your data should still be there.
- **At least 4 functions you wrote yourself** (Module 6), each doing one clear job — not one giant `main`.
- **Pointers used with intention** (Module 7) somewhere real — e.g. passing a struct by pointer to modify it, not just because it's required. Be ready to explain why you used a pointer where you did.
- **Arrays and/or strings** (Module 5) for any collection of data or text processing.
- **Control structures and a menu-driven loop** (Module 4) — the program should run continuously, present options, and only exit when the user chooses to.
- **A project `README.md`** in your team's repo explaining: what the program does, how to compile and run it, and a one-paragraph description of who worked on what.
- **No compiler warnings** with `gcc -Wall yourfile.c -o yourprogram`. Warnings are gcc telling you about likely bugs — clear them all.

4. Choose a project

Pick one. All of them are scoped to fit the requirements above with room for your own ideas on top.

1. **Student Grading & Report Card System** — store students (struct: name, ID, array of scores), compute averages/grades, save/load the class roster to a file. *Stretch*: rank students, export a formatted report per student.
2. **Library / Inventory Management System** — track items (struct: title, ID, quantity, checked-out status), check items in/out, persist the catalog to a file. *Stretch*: search by keyword, track due dates.
3. **Contact Book / Phone Directory** — add, search, edit, and delete contacts (struct: name, phone, email), saved to a file. *Stretch*: sort alphabetically, group by category.
4. **Quiz Game Engine** — load a question bank from a file, ask questions, score the player, save high scores. *Stretch*: multiple difficulty levels, a timer.
5. **Simple Bank Account Simulator** — deposit, withdraw, check balance, with a persisted transaction log (struct per transaction). *Stretch*: multiple accounts, interest calculation, transaction history search.
6. **Text-Based Adventure Game** — rooms and choices driving the player through a small story, with save/load progress. *Stretch*: an inventory system using structs, multiple endings.
7. **Personal Expense Tracker** — log expenses (struct: category, amount, date) to a file, show totals and simple statistics. *Stretch*: monthly summaries, a simple budget-vs-actual comparison.

Have your own idea? Propose it to your instructor — as long as it can satisfy every requirement in §3, it's fair game.

5. Suggested timeline

Sprint	Focus	Git evidence expected
Sprint 0	Team formed, project chosen, <code>structs</code> designed on paper	An initial commit with your struct definitions and a plan in

Sprint	Focus	Git evidence expected
	first	the README
Sprint 1	Core data operations working (add/search/edit) in memory, no file I/O yet	Several small commits, each compiling cleanly
Sprint 2	File I/O: save and load your data correctly	A commit where you can close and reopen the program without losing data
Sprint 3	Menu loop, polish, edge cases (empty list, bad input)	Commits addressing specific bugs/edge cases you found
Final	Clean <code>-Wall</code> build, finished README, last push before deadline	Your final commit <i>is</i> your submission

Commit at the end of every session you work, even if it's not finished — a commit history of one giant commit the night before the deadline is a red flag, not a shortcut.

6. How this gets assessed

- **Correctness** — does it do what it claims, including edge cases (empty input, invalid menu choice, saving/loading correctly)?
- **Use of the module concepts** — structs, pointers, functions, arrays, and file I/O used appropriately, not bolted on to satisfy a checklist.
- **Code organization** — logically split into functions, consistent style, comments where the *why* isn't obvious.
- **Git history** — regular, meaningful commits from every team member under their own name.
- **Documentation** — a README a stranger could use to build, run, and understand your project without asking you anything.

Try it yourself

1. Confirm your team (or solo status) in GitHub Classroom.
2. Clone your team's repo.
3. Pick a project from §4 (or propose your own).
4. Work through the sprints in §5, committing and pushing regularly.
5. Submit by pushing your final commit before the deadline — there is no separate submission step.

You've built something real. Put the repo link on your CV.